# L'Architecture Client-Serveur et le Protocole HTTP

**Thème principal :** Communication Web, Protocole HTTP Stateless, Structure des Messages et Diagnostic des Codes de Statut

**Synthèse exécutive :** Ce cours présente les fondements fondamentaux du Web en décortiquant le fonctionnement de l'architecture client-serveur et les règles de communication imposées par le protocole HTTP (HyperText Transfer Protocol). L'étudiant y apprendra comment un navigateur (client) initie de façon asymétrique des requêtes textuelles pour obtenir des ressources auprès d'un serveur web (Apache, Nginx). La séance détaille la nature 'stateless' (sans état) du protocole et ses solutions de contournement (cookies/sessions), la structure normalisée des messages HTTP (lignes de commande/statut, en-têtes, corps), le comparatif strict entre les méthodes GET et POST ainsi que leur impact sur la sécurité, et enfin la grille de lecture des codes de statut HTTP (2xx, 3xx, 4xx, 5xx) indispensables au débogage d'applications web.

## 1. L'Architecture Client-Serveur et la Nature Stateless de HTTP

L'architecture client-serveur est le modèle conceptuel régissant les échanges sur le Web. Contrairement aux architectures Pair-à-Pair (P2P) où les nœuds sont équivalents, ce modèle est strictement asymétrique. Le client (un navigateur web tel que Firefox, Chrome ou Safari) est l'élément actif : il formule et envoie une requête HTTP à travers le réseau réseau IP. Le serveur web (un démon applicatif distant tel qu'Apache, Nginx ou IIS) est l'élément passif : il écoute en continu sur un port réseau dédié (port 80 pour HTTP en clair, port 443 pour HTTPS chiffré). Lorsqu'il reçoit une requête, il la traite — ce qui peut nécessiter la lecture de fichiers locaux ou l'exécution de scripts interrogeant une base de données — puis retourne une réponse HTTP au client. Une propriété fondamentale du protocole HTTP est sa nature 'stateless' (sans état). Cela signifie que chaque cycle requête-réponse est totalement indépendant, isolé et amnésique. Une fois la réponse envoyée, le serveur clôt ou réinitialise le canal de communication sans conserver en mémoire l'identité ou les actions passées du client. Si ce fonctionnement simplifie la conception des infrastructures et offre une scalabilité optimale (le serveur n'a pas à maintenir des millions d'états en mémoire vive), il interdit nativement le suivi d'un utilisateur. Sans mécanisme complémentaire, un panier d'achat se viderait à chaque clic et l'utilisateur devrait ressaisir ses identifiants à chaque page. Pour contourner cette limite native, on utilise des cookies de session : lors de la première connexion, le serveur génère un identifiant unique (Session ID) transmis dans les en-têtes de réponse. Le navigateur stocke ce cookie et le réémet automatiquement dans l'en-tête de chaque requête ultérieure, permettant au serveur de reconstituer le contexte de l'utilisateur.

- Asymétrie Client-Serveur : Le client (navigateur) prend toujours l'initiative d'émettre une requête ; le serveur attend passivement et renvoie une réponse.
- Protocole Stateless (Sans état) : Absolutisme du protocole HTTP qui ne conserve aucune mémoire des transactions précédentes.
- Gestion de Session (Cookies) : Mécanisme d'en-tête permettant d'injecter un identifiant de session pour simuler un état persistant.

***Cas concret enseignant :*** *Le professeur compare le protocole HTTP à un guichetier amnésique : chaque fois que vous vous présentez devant lui, même si c'était il y a deux secondes, il vous traite comme un inconnu total. Pour qu'il se souvienne de vous, vous devez lui montrer votre badge (le cookie de session) à chaque phrase. De plus, lors d'une simple visite sur le site de l'IUT de Bayonne, le navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes HTTP simultanées pour récupérer le fichier HTML de base, puis chaque feuille de style CSS, script JavaScript et image associée.*

## 2. Structure Interne des Messages HTTP et Comparatif GET vs POST

Tous les messages transitant via le protocole HTTP sont des blocs textuels lisibles, structurés de manière stricte. Une requête HTTP comprend quatre parties : 1. La ligne de requête : Elle contient le verbe HTTP (méthode comme GET ou POST), l'URI (Uniform Resource Identifier, le chemin absolu de la ressource demandée) et la version du

protocole (ex: HTTP/1.1). 2. Les en-têtes (Headers) : Paires clé-valeur fournissant des métadonnées contextuelles. L'en-tête 'Host' est obligatoire en HTTP/1.1 pour spécifier le nom de domaine cible (permettant le Virtual Hosting). L'en-tête 'User-Agent' identifie le système et le navigateur du client. 3. Une ligne vide (CRLF) : Obligatoire, elle sert de séparateur strict entre les en-têtes et le corps. 4. Le corps (Body) : Optionnel en GET, il contient les données brutes envoyées au serveur lors d'une méthode POST (données de formulaire, JSON, fichiers). Le choix entre les méthodes GET et POST est un enjeu critique de conception et de sécurité : - La méthode GET est conçue exclusivement pour consulter ou récupérer des ressources sans altérer l'état du serveur (méthode dite 'sûre' et 'idempotente'). Ses paramètres sont accolés à l'URL sous forme de chaîne de requête (Query String: `?champ1=valeur1&champ2;=valeur2`). GET est limitée en taille (environ 2048 caractères) et ne doit jamais transmettre de données sensibles. - La méthode POST est conçue pour soumettre des données destinées à modifier l'état du serveur (création d'un compte, validation d'une commande). Les données sont encapsulées dans le corps (body) du message HTTP, rendant les paramètres invisibles dans la barre d'adresse et levant la restriction de taille d'URL.

- Format Textuel Normalisé : Un message HTTP est structuré en clair avec une ligne de commande/statut, des en-têtes (headers), une ligne vide obligatoire et un corps (body).
- Méthode GET : Extraction et consultation de ressources, méthode sûre et idempotente transmettant ses paramètres dans l'URL.
- Méthode POST : Soumission de données modifiant l'état du serveur, encapsulant ses variables dans le corps du message sans limitation de longueur d'URL.

***Cas concret enseignant :*** *Le professeur alerte vivement sur l'utilisation absurde de GET pour envoyer un formulaire de connexion : transmettre un mot de passe en GET l'affiche directement en clair dans la barre d'adresse de l'écran, le sauvegarde dans l'historique du navigateur et l'enregistre en clair dans les fichiers de logs du serveur web. C'est une vulnérabilité critique exposant l'utilisateur au 'shoulder surfing' (regard par-dessus l'épaule).*

## 3. Décodage et Classification des Codes de Statut HTTP

Chaque réponse émise par un serveur web débute par une 'ligne de statut' (ex: `HTTP/1.1 200 OK`). Le code numérique à trois chiffres permet au client applicatif de comprendre immédiatement le résultat du traitement sans avoir à analyser l'intégralité du corps de la réponse. Les codes sont répartis en cinq grandes familles : - 1xx (Information) : Relevés d'information ou transfert en cours. - 2xx (Succès) : La requête a été reçue, comprise et acceptée avec succès (ex: `200 OK` pour une requête honorée, `201 Created` après une création de ressource). - 3xx (Redirection) : Indique que le client doit effectuer des actions supplémentaires pour accéder à la ressource (ex: `301 Moved Permanently` pour une redirection définitive, `304 Not Modified` exploitant le cache du navigateur). - 4xx (Erreurs du Client) : Indique que la requête contient une anomalie ou ne peut pas être exécutée en l'état. La responsabilité incombe au client. Exemples : `401 Unauthorized` (authentification requise), `403 Forbidden` (droits insuffisants), `404 Not Found` (URI ciblant une ressource inexistante sur le serveur). - 5xx (Erreurs du Serveur) : Indique que la requête du client était valide, mais que le serveur a rencontré une défaillance interne lors du traitement. La responsabilité réside exclusivement sur le code backend ou l'infrastructure. Exemples : `500 Internal Server Error` (crash d'un script PHP/Node.js, erreur de syntaxe), `503 Service Unavailable` (serveur saturé ou en maintenance).

- Ligne de Statut de Réponse : Première ligne d'une réponse HTTP contenant la version du protocole, un code numérique à trois chiffres et sa description textuelle.
- Familles de Codes (1xx à 5xx) : Classification logique du résultat de la requête (2xx Succès, 3xx Redirection, 4xx Erreur Client, 5xx Erreur Serveur).
- Diagnostic de Débogage : Frontière stricte entre la responsabilité de l'application cliente (4xx) et le dysfonctionnement de l'infrastructure/code backend (5xx).

***Cas concret enseignant :*** *Le professeur raconte une mésaventure d'envergure : à ses débuts, une mauvaise configuration de son script PHP faisait planter la connexion à la base de données lors de la validation d'un formulaire. Le serveur renvoyait systématiquement une erreur 500 Internal Server Error. Les utilisateurs, effrayés par cette page brute, pensaient que le site avait été piraté alors qu'il s'agissait simplement d'un point-virgule manquant dans son code backend.*

## ■ Questions & Interventions des Étudiants en Classe

**Question étudiant :** Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
**Réponse du professeur :** Pour un formulaire de connexion, il faut impérativement utiliser la méthode POST. Avec GET, vos identifiants (login et mot de passe) sont directement collés dans l'URL sous la forme `?login=matei&pass;=1234`. Ils apparaissent en clair sur votre écran (risque de shoulder surfing), restent enregistrés dans l'historique du navigateur et sont stockés en clair dans les logs d'accès du serveur web. Avec POST, les identifiants sont dissimulés dans le corps (body) de la requête HTTP, préservant la barre d'adresse et l'historique (bien qu'il faille toujours utiliser HTTPS pour chiffrer la communication sur le réseau).

## ■ Exercice & Démonstration d'Application

**Énoncé :** Analyser la requête HTTP brute suivante. Identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer le comportement du serveur avec le code de retour exact si la ressource sollicitée n'existe pas sur le disque : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

**Résolution pas à pas :**
1. Analyse de la ligne de requête ('GET /images/logo.png HTTP/1.1') : - Méthode HTTP : GET. Il s'agit d'une demande d'extraction/récupération de ressource sans modification de l'état du serveur. - Ressource demandée (URI) : '/images/logo.png'. C'est le chemin d'accès absolu au fichier sur le serveur. - Version du protocole : HTTP/1.1. 2. Analyse de l'en-tête Host ('Host: iutbayonne.univ-pau.fr') : - Cet en-tête spécifie le nom de domaine du serveur destinataire. Il est obligatoire en HTTP/1.1 pour permettre au serveur Web d'effectuer du 'Virtual Hosting' (hébergement de plusieurs sites web distincts sur une même adresse IP). 3. Analyse de l'en-tête User-Agent ('User-Agent: Mozilla/5.0') : - Transmet la signature du logiciel client (navigateur) effectuant la requête au serveur. 4. Scénario d'absence de la ressource : - Si le fichier 'logo.png' n'existe pas dans le répertoire '/images/' du serveur web, la requête pointe vers un emplacement invalide. - L'erreur provenant de la formulation d'une mauvaise URL par le client, la responsabilité lui incombe. - Le serveur doit renvoyer une réponse HTTP contenant un code de statut de la classe 4xx, précisément le code '404 Not Found'.

**Résultat : Méthode : GET | Ressource demandée : /images/logo.png | Hôte cible : iutbayonne.univ-pau.fr | Code de réponse en cas d'absence : 404 Not Found (Erreur de la responsabilité du Client).**

## ■ Auto-Évaluation Flash d'Examen

**Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP et quelle est sa conséquence directe sur la gestion des sessions ?**

*R : Un protocole 'stateless' (sans état) ne conserve aucune mémoire des transactions précédentes. Chaque échange requête-réponse est isolé et amnésique. Pour maintenir un état persistant (utilisateur connecté, panier d'achat), il faut utiliser des mécanismes complémentaires comme les cookies, transmis par le navigateur à chaque requête avec un identifiant de session.*

**Q : Quelle est la différence fondamentale de responsabilité entre une erreur HTTP de la série 4xx et une erreur de la série 5xx ?**

*R : Une erreur 4xx indique une anomalie dont la responsabilité incombe au client (URL mal saisie, ressource inexistante 404, absence d'autorisation 401/403). Une erreur 5xx indique un dysfonctionnement interne dont la responsabilité incombe au serveur (crash de script backend, panne de base de données 500), alors que la requête du client était valide.*

**Q : Pourquoi la méthode HTTP GET ne doit-elle jamais être employée pour transmettre un mot de passe ?**

*R : Parce que GET transmet ses paramètres directement dans l'URL. Le mot de passe apparaît en clair dans la barre d'adresse (risque de shoulder surfing), reste stocké dans l'historique du navigateur client, est conservé dans les logs des serveurs proxy/web et peut être divulgué via l'en-tête Referer.*